

# Optimizing Nextflow-based Software on Shared HPC Resources: A Case Study with `make_lastz_chains`

Nil Tianchen Mu<sup>1</sup>, William Dizon<sup>1</sup>, Glen Otero<sup>1</sup>, Torey Battelle<sup>1</sup>

*Research Technology Office*

*Arizona State University*

Tempe, Arizona, USA

nilmu@asu.edu, wdizon@asu.edu, gotero3@asu.edu, tbattell@asu.edu

**Abstract**—Nextflow is a widely adopted workflow manager in the bioinformatics community, known for its scalability, portability, and reproducibility. However, on shared HPC clusters that use the Slurm job scheduler and the Fairshare score to record historical resource usage and determine current job queuing positions, individual Nextflow job submissions negatively impact user Fairshare scores and lead to extended queue wait times. In this paper, we share practical observations from supporting researchers running the Hiller Lab `make_lastz_chains` pipeline, which uses Nextflow to orchestrate genome alignment with LASTZ and related UCSC tools, on the Arizona State University supercomputers. We identify key challenges and solutions regarding scheduling, Fairshare impact, and capturing Slurm errors. We would like to share these observations and practical considerations with researchers, RSEs, and HPC system administrators in order to improve management of Nextflow workflows on shared HPC resources, foster more efficient resource utilization, and a smoother user experience.

**Index Terms**—Nextflow, Slurm, Fairshare, `make_lastz_chains`, High Performance Computing.

## I. INTRODUCTION

This paper stems from our recent experience supporting researchers utilizing the Nextflow-based `make_lastz_chains` software ([https://github.com/hillerlab/make\\_lastz\\_chains](https://github.com/hillerlab/make_lastz_chains)) [1] on the Arizona State University supercomputers [2]. We observed significant performance and resource management challenges, mainly related to how Nextflow [3] interacts with the High Performance Computing (HPC) job scheduler Slurm [4] by default. Our goal is not to critique the `make_lastz_chains` software or Nextflow itself, but to share our observations and offer practical insights to other Research Software Engineers (RSEs), HPC system administrators, researchers, and students so that they may achieve a better starting point when deploying similar Nextflow-based tools in a shared HPC environment.

High-throughput genomic analysis pipelines increasingly rely on workflow engines like Nextflow to manage complex,

multi-stage computations. Although techniques employing Slurm Job Arrays [5] are an alternative when implementing such pipelines, Nextflow stands out due to its steady learning curve, abstraction, portability, reproducibility, scalability, and containerization support [3].

The software `make_lastz_chains`, developed by the Michael Hiller Lab [1], is one such pipeline designed to perform highly parallelized genome alignment and chaining using LASTZ [7] and related UCSC tools [6], with LASTZ parameters tuned by the Hiller Lab as default settings. LASTZ (Large-Scale Genome Alignment Tool) is primarily designed for whole genome alignments between two large, often chromosome-scale, DNA sequences. It excels at comparing entire genomes of different species, even heavily divergent ones, to identify orthologous regions, conserved synteny (order of genes), and large-scale evolutionary events like inversions and translocations. It is a powerful tool for comparative genomics [7].

The key concepts related to our investigation are “Masking” and “Chaining”:

- Masking refers to identifying and marking repetitive sequences and low-complexity regions within DNA sequences before performing the alignment. Masking is a crucial pre-processing step that significantly impacts the efficiency and accuracy of the alignment. Masking needs to be applied to the input data before starting the `make_lastz_chains` pipeline [1].
- A chain is essentially a sequence of non-overlapping local alignment blocks (often initially gapless) found between two genomic sequences (a target and a query). The chaining process involves merging overlapping raw local alignments and joining compatible, co-linear alignments to form longer, more significant alignment structures with higher alignment scores. In this process, heavy computation and parallelization are needed [7].

The specific project applies `make_lastz_chains` to align large genome sequences; ranging from 2GB to 9GB in file size. Since `make_lastz_chains` parallelizes the chaining process with Nextflow, and any failed child job will affect the final

William Dizon provided analysis and guidance about Slurm job queuing, discovered and implemented the public partition solution.

Glen Otero and Torey Battelle supervised and reviewed the work.

output chains, it cannot tolerate any failure of parallelized Nextflow tasks or “child jobs”. Thus, it requires sufficient computational resources and extended run times on HPC clusters. Slurm is one of the major HPC job schedulers, and it uses Fairshare to allocate resources equitably among users. While such policies promote fairness, they also introduce trade-offs that impact workflows with large numbers of small, independent jobs. Given the vast number of Nextflow child jobs that `make_lastz_chains` spawns, working around a researcher’s limited Fairshare score and the increasing wait times for child jobs are the two main challenges encountered.

This use case offers a unique opportunity to support the research community in implementing complex Nextflow-based computational tools in shared HPC environments.

## II. SYSTEM SPECIFICATIONS

### A. HPC Environment

- HPC Clusters:
  - Sol Supercomputer: 21,000+ CPU and 290+ GPU cores using the modern AMD Epyc architecture, Rocky Linux 8.10.
  - Phoenix Supercomputer: 17,200+ CPU and 360+ GPU cores using heterogeneous Intel-based architecture, Rocky Linux 8.10.
- Scheduler: Slurm 23.11.1 on Sol, 24.11.3 on Phoenix
- User Fairshare Policy: A user’s Fairshare halves for every 10,000 core-hour equivalents (CHE) of usage on ASU supercomputers [8].

### B. Software Stack

- Lmod 8.7.55 [9]
- Spack 0.21.0.dev0 on Sol and 0.21.2 on Phoenix [10]
- Nextflow 23.04.3 on Sol and 23.10.0 on Phoenix, installed by Spack
- OpenJDK 17.0.3\_7, installed by Spack
- `make_lastz_chains` 2.0.8 is installed in a Python environment provided by the `mamba` module using `mamba 1.5.9` on both Sol and Phoenix.

## III. IMPLEMENTATION

Figure 1 illustrates the major steps in the `make_lastz_chains` pipeline [1]. It involves three waves of Nextflow processes, each requiring a full completion rate for its child jobs. If a Nextflow process fails, the whole workflow will be aborted. The steps that happen in the main job are I/O intensive but only use a fraction of the computational time compared to the Nextflow processes.

Figure 2 displays the relationship between the main job and the Nextflow processes including Slurm job details. The main job must persist long enough to allow all the Nextflow child jobs for all three Nextflow steps in the pipeline to finish. However, since the majority of computations reside in the child jobs, the main job only needs 1 CPU and around 10 to 20 GiB of memory to handle the associated data I/O tasks. The spawned Nextflow child jobs are lightweight and without multi-threading capability and therefore not CPU or memory

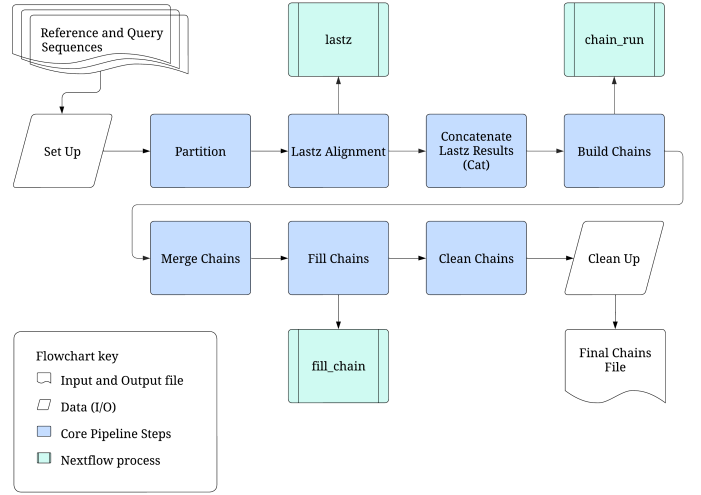


Fig. 1. High-level workflow of `make_lastz_chains`

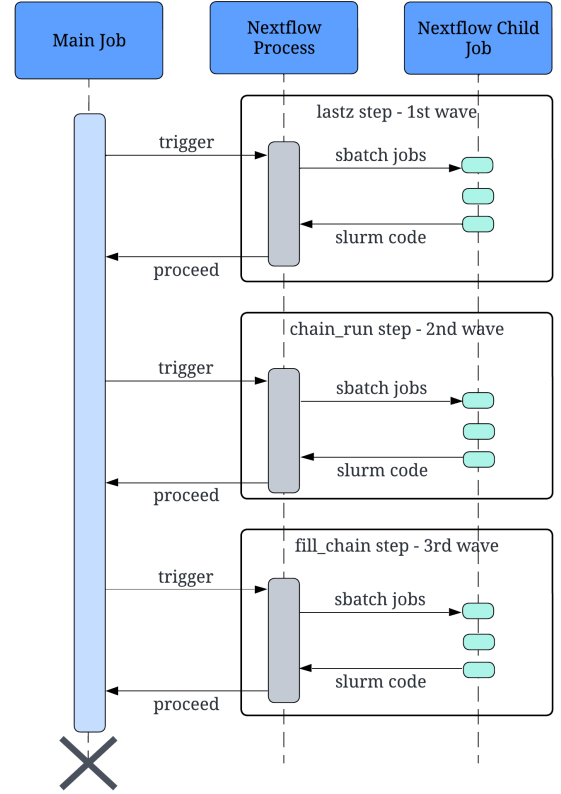


Fig. 2. Sequence Diagram of `make_lastz_chains`

intensive [1]. After rounds of trial and error, 1 CPU, 4 GiB of memory, and 30 minutes to 1 hour job time were allocated for each child job.

The resources for the main job are listed as the sbatch parameters of the sbatch job script of the main job. The resources for the child jobs are hard-coded in the Nextflow configuration file, which is generated by the `nextflow_wapper.py` script of `make_lastz_chains` code base [1]. Using

the sbatch command to submit the main job is the only job submission action done on the user side. The #SBATCH --export=NONE line should not be included in the sbatch script of the main job because it prevents the child jobs from inheriting the PATH variables of the Python environment and thus prevents the child jobs from using the executables in make\_lastz\_chains.

Below is an example of an sbatch script for the main job.

```
#!/bin/bash
#SBATCH -t 3-0
#SBATCH --mem=20G
#SBATCH -p example_partition
#SBATCH -q example_qos
#SBATCH -o chains_%j.out
#SBATCH -e chains_%j.err

module load nextflow-23.10.0
module load openjdk-17.0.3_7
module load mamba
source activate make_lastz_chains-2.0.8

input_dir="/path/to/input"
working_dir="/path/to/work"
export PATH=/path/to/\
make_lastz_chains-2.0.8/bin:$PATH
cd /path/to/make_lastz_chains/

python make_chains.py\
-f --project_dir $working_dir/run1\
--cluster_executor slurm\
--cluster_queue example_partition\
--seq1_chunk 40000000 --seq2_chunk 40000000\
--chaining_memory 20\
--job_time_req 00:30:00\
$ref_species $query_species\
$input_dir/ref.fasta $input_dir/query.fasta
```

Listing 1. Sample Main Sbatch Job

This design shrewdly exploits parallelization in an HPC application. Instead of executing all the computational work sequentially in one enormous job, which would be impossible to allocate on a shared Slurm cluster, it decomposes the enormous aggregate demands of CPU, memory, and job time into many small sbatch jobs.

#### IV. OBSERVATIONS AND DISCUSSIONS

##### A. First Challenge: Dynamic Resource Allocation

As described above, the resources for each Nextflow child job are hard-coded. We did not find one set of sbatch parameters that could accommodate the varied resource requirements of all the child jobs, and this caused many time-outs and out-of-memory (OOM) errors. However, if we increased the resources for all the child jobs, the queuing time would significantly increase for all the child jobs spawned later. We found that Nextflow has an innovative solution for this kind of scenario: resubmitting the failed child job and requesting more resources. Our dynamic resource allocation code modifications [11] added to the nextflow\_wapper.py script in the make\_lastz\_chains code base are listed below:

```
...
def dump_to_file(self):
    ...
    f.write("process.memory={4.GB*task.attempt}\n")
    f.write("process.time={0.5.hour*task.attempt}\n")
    f.write("process.cpus=1\n")
    f.write(f"process.queue='{self.queue}'\n")
    f.write(f"executor.queueSize=1000\n")
    f.write(f"process.maxRetries=10\n")
    f.write(f"process.errorStrategy='retry'\n")
    f.write(f"process.maxErrors='-1'\n")
    ...
```

Listing 2. Nextflow Configuration

With these modifications, the first retry of a failed child job will have 8GiB of memory and 1 hour of job time, the second retry will have 16GiB and 1.5 hours of job time, etc. [11]. The maximum number of opportunities for a child job to run is 10 times, and the maximum count of failed child jobs (including the retried ones) that the Nextflow process can tolerate is set to infinite (line 11), so all the child jobs would have multiple chances to succeed. Nextflow has many other helpful functionalities, such as “execution cache and resume” [16]. We are continuing to investigate other optimization opportunities.

##### B. Second Challenge: Long Job Queuing Time

Since a single Nextflow child job in a Nextflow process is submitted as an independent sbatch job, its queuing position considers the user’s Fairshare score at the moment of its submission time, and its job pending time is unrelated to other child jobs. Thus, the user’s Fairshare will decrease gradually as Nextflow spawns more child jobs with gradually delayed start times. This, in turn, causes the main job to wait longer and longer to proceed to the next step. At the beginning of our investigation, this scenario caused many time-out errors for the main jobs, especially on the Sol supercomputer, which is constantly in high demand.

The Sol and Phoenix supercomputers follow the Condominium Model [12], meaning they have both public and private-owned nodes on the cluster. Both supercomputers have a partition called HTC (high-throughput computing), which allows non-preemptable jobs for up to 4 hours, covering all of the public nodes and most of the private nodes. Since the HTC partition includes almost all the nodes on the cluster and has a very short job time limit, jobs on the HTC partition often experience the shortest wait time compared to other partitions. Thus the HTC partition was chosen on both clusters to run make\_lastz\_chains jobs initially.

However, we noticed that as the Fairshare score quickly decreases, even on the HTC partition, the jobs from the private node owners with a much higher Fairshare score would almost completely block the child jobs from getting allocated. Later, we created a new partition called “public” and asked only users without private nodes to use this partition. This change did not help us much on Sol, but made a significant difference on Phoenix, because most of the users on Phoenix

are private node owners. This finding was reported to the Slurm SchedMD team and this change is being considered for permanent implementation on Phoenix.

After the above change, child jobs could then be allocated almost immediately after submission, spreading across the over 300 CPU nodes in the Phoenix public partition (over 8400 CPU cores and 36,600GiB memory). Figure 3 below is a screenshot of the Phoenix public partition on our Phoenix status dashboard [15] taken on May 8th, 2025 when three main jobs were running and progressing through the Nextflow steps. Most of the CPUs in the public partition were allocated and the nodes showing orange or red.

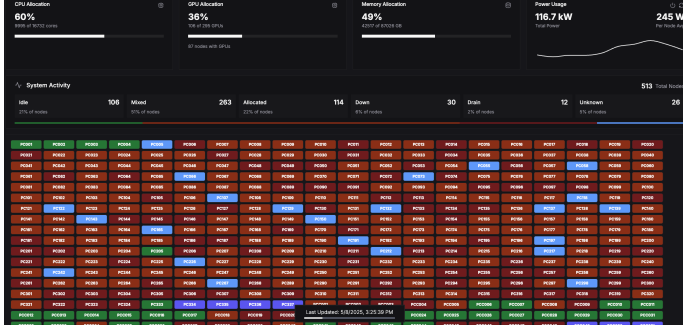


Fig. 3. Fully Occupied Public Partition on Phoenix

However, in the interest of avoiding an overload of Nextflow with too many child jobs, we limited the number of main jobs submitted. Too many child jobs would also create a scheduling bottleneck. As the number of jobs exceeds the capacity of the public partition, the child jobs would start competing with each other. Slurm would spend time reconciling the competition, thus adding to the pending time of child jobs. That is also why we kept the `executor.queueSize` in the Nextflow Configuration set to 1000, which is the default value [13]. This number means that at any given time, one Nextflow process will hold up to 1000 sbatch jobs (in various states). We have experimented increasing it to 3000, but we found that our Nextflow instance could not reach the upper limit. We are working on collecting more data from the Slurm logs to quantify this observation.

### C. Third Challenge: Child Job Completion

As we worked through the dataset with the researchers, we encountered more issues around the Nextflow retry mechanism and Slurm.

As shown in Figure 4, sometimes Nextflow would fail to trigger a retry for a failed child job. This is problematic since the main job requires all the child jobs in a Nextflow step to succeed or it will not proceed to the next steps and will crash. We observed that Nextflow catches the error code “140” (insufficient resources such as time-out and OOM) very well. However, for some cases, Slurm would send a `0:0` signal for some child jobs that were stuck in the “completing” state, and Nextflow could not trigger retries for that signal. Moreover, if Slurm does not send any error signal for a crashed child job,

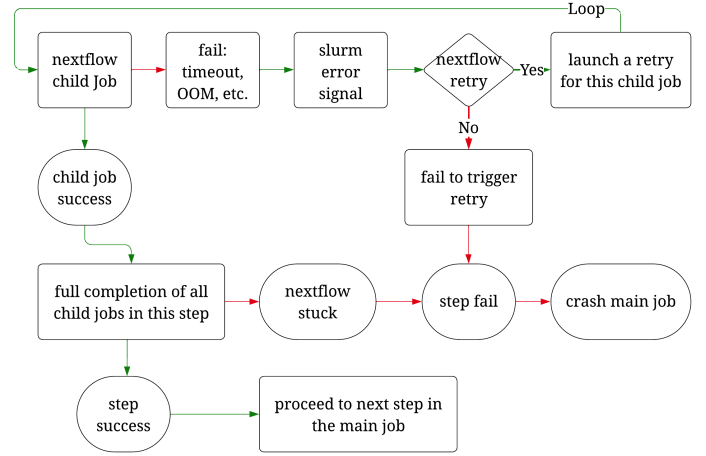


Fig. 4. Nextflow child job retry flowchart

it will not get a retry chance. We are considering adding a “Derived Exit Code” for the child jobs with the `0:0` error code, with the `sjobexitmod` command from Slurm, and to see if Nextflow retries can be triggered by this additional Slurm error signal.

Later, we discovered that some of the nodes in the Phoenix public partition experienced hardware-related failures, and the child jobs that landed on the impaired nodes were not retried. This lesson indicates that before launching some time-consuming large-scale Nextflow jobs, some partition-wide node health checks would be helpful.

### D. Fourth Challenge: The Gap Between Test and Real Data

The above challenges were unknown to us when we first installed the `make_lastz_chains` Python environment. Since the small test case included in the software package worked well [1], we handed the installation over to the researchers with little knowledge of how a large unique dataset would fare. As we have documented here, the researcher’s dataset introduced additional complications. This clearly revealed how small or synthetic test cases sometimes cannot capture the full picture.

During our investigations the LASTZ parameters were kept unchanged while focusing on the Nextflow configurations. We later learned that some Nextflow errors could be addressed by adding more masking to the input dataset or changing the default LASTZ parameters. These methods require more domain knowledge, and we have been working closely with our researchers to explore these possibilities.

## V. CONCLUSION

Nextflow enables scalable and reproducible genomics pipelines, but its default job management strategy can create challenges on shared HPC systems with Fairshare scheduling. Our experience supporting `make_lastz_chains` illustrates the importance of tuning job granularity and cluster-aware configuration. By sharing these observations and practices, we hope to help equip fellow RSEs, HPC system admins, researchers, and learners with better starting points, fostering more efficient and equitable use of HPC resources.

## ACKNOWLEDGMENT

We acknowledge Research Computing of the Research Technology Office (RTO) at Arizona State University [14] for providing computing, storage, etc. resources that have contributed to the research results reported within this paper. Dr. Gil Speyer from the RTO Computational Research Accelerator team helped with reviewing the work and this paper.

We acknowledge the generous help and patience from Simone Gable from the Knowledge Enterprise Global Futures Laboratory, Arizona State University and other researchers. We appreciate the opportunity to work with them on implementing `make_lastz_chains` on ASU supercomputers.

We acknowledge the developers of `make_lastz_chains`, especially Michael Hiller from the Senckenberg Research Institute and Goethe-University, Germany for helping with testing, discussing technical details, and providing recommendations for various parameters.

## REFERENCES

- [1] BM Kirilenko, C Munegowda, E Osipova, D Jebb, V Sharma, M Blumer, A Morales, AW Ahmed, DG Kontopoulos, L Hilgers, K Lindblad-Toh, EK Karlsson, Zoonomia Consortium, M Hiller. "Integrating gene annotation with orthology inference at scale.," *Science*, 380, 2023
- [2] "Welcome to ASU Research Computing," <https://asurc.atlassian.net/wiki/spaces/RC/overview> (accessed May 15, 2025)
- [3] P. Di Tommaso, et al. "Nextflow enables reproducible computational workflows," *Nature Biotechnology* 35, 316–319 (2017) doi:10.1038/nbt.3820
- [4] Yoo, A.B., Jette, M.A., Grondona, M. (2003). SLURM: Simple Linux Utility for Resource Management. In: Feitelson, D., Rudolph, L., Schwiegelshohn, U. (eds) "Job Scheduling Strategies for Parallel Processing," JSSPP 2003. Lecture Notes in Computer Science, vol 2862. Springer, Berlin, Heidelberg. [https://doi.org/10.1007/10968987\\_3](https://doi.org/10.1007/10968987_3)
- [5] "Slurm Workload manager," Slurm Workload Manager - Job Array Support," [https://slurm.schedmd.com/job\\_array.html](https://slurm.schedmd.com/job_array.html) (accessed May 15, 2025)
- [6] Perez G, Barber GP, Benet-Pages A, et al. "The UCSC Genome Browser database: 2025 update," *Nucleic Acids Res.* 2025;53(D1):D1243-D1249. doi:10.1093/nar/gkae974
- [7] R.S. Harris, "Improved pairwise alignment of genomic DNA," Ph.D. Thesis, The Pennsylvania State University, 2005
- [8] Jason Yalim. "Toward Dynamically Controlling Slurm's Classic Fairshare Algorithm," In Practice and Experience in Advanced Research Computing 2020: Catch the Wave (PEARC '20). Association for Computing Machinery, New York, NY, USA, 538–542. 2020. <https://doi.org/10.1145/3311790.3399628>
- [9] R. McLay, K. W. Schulz, W. L. Barth, and T. Minyard, "Best practices for the deployment and management of production HPC clusters," In State of the Practice Reports, SC11, pages 9:1–9:11, Nov. 2011. doi.acm.org/10.1145/2063348.2063360.
- [10] Todd Gamblin, Matthew P. LeGendre, Michael R. Collette, Gregory L. Lee, Adam Moody, Bronis R. de Supinski, and W. Scott Futral. "The Spack Package Manager: Bringing Order to HPC Software Chaos," In Supercomputing 2015 (SC'15), Austin, Texas, November 15–20 2015. LLNL-CONF-669890.
- [11] "Error handling and troubleshooting," Nextflow Troubleshooting," [https://training.nextflow.io/2.1/basic\\_training/debugging/#dynamic-resourcesallocation](https://training.nextflow.io/2.1/basic_training/debugging/#dynamic-resourcesallocation) (accessed May 15, 2025)
- [12] James B. Bottum, Ruth Marinshaw, Henry Neeman, James Pepin, and J. Barr von Oehsen. "The condo of condos," In Proceedings of the Conference on Extreme Science and Engineering Discovery Environment: Gateway to Discovery (XSEDE '13). Association for Computing Machinery, New York, NY, USA, Article 62, 1–2. 2013. <https://doi.org/10.1145/2484762.2484775>
- [13] "Configuration options," Configuration options - Nextflow documentation," <https://www.nextflow.io/docs/latest/reference/config.html> (accessed May 15, 2025)
- [14] Jennewein, Douglas M. et al. "The Sol Supercomputer at Arizona State University," In Practice and Experience in Advanced Research Computing (pp. 296–301). Association for Computing Machinery, 2023.
- [15] Johnathan Lee. "HPC Dashboard," <https://github.com/thediy maker/slurm-node-dashboard> (accessed May 15, 2025)
- [16] "Execution cache and resume," Nextflow Troubleshooting," [https://training.nextflow.io/2.1/basic\\_training/cache\\_and\\_resume/](https://training.nextflow.io/2.1/basic_training/cache_and_resume/) (accessed May 15, 2025)